

People Pulse — Full MVP Spec

Promptathon build · 1-day scope · Derived from "Global People Intelligence & Activity Dashboard Agent" brief (Jim Gordon)

Stack decisions locked in for this build: **Python** for data wrangling, **Azure AI Foundry** for model calls, **Streamlit** for the dashboard (rationale in §4).

1. Purpose

A working, demoable slice of the full brief: a unified dashboard covering the complete employee lifecycle, with an AI narrative layer and a natural-language query box on top, built in one day.

This version corrects the earlier draft's gap — it now covers all eight lifecycle tracking areas named in the brief, not a subset.

2. Scope

In scope — all 8 named tracking areas

#	Brief area	Source sheet(s)	Dashboard treatment
1	ER cases	ER_Case_Log	Tile: open/closed by severity, avg resolution days, load by handler
2	Onboarding	Onboarding_Tracker	Tile: stage completion funnel, avg induction feedback score
3	Performance reviews (incl. 3rd years)	Performance_Reviews	Tile: rating distribution by review type, trend over cycles
4	Bench	Bench_Tracking	Tile: headcount on bench, risk flag breakdown, avg days on bench
5	Engagement	Engagement_Survey	Tile: satisfaction/eNPS trend across survey waves
6	Pay reviews	Pay_Reviews	Tile: pending/approved/rejected, £ budget impact
7	Probation tracking	Probation_Tracking	Tile: pass/fail/extended rate
8	FTC tracking	FTC_Tracking	Tile: contracts nearing expiry, renewal outcomes

Plus, from the brief's "People initiatives" line:

- **Touchpoints/events** (Touchpoints_Events) and **team workload** (PeopleTeam_Activity_Log) as a ninth summary tile — team activity/capacity view

Core cross-cutting filter: **Academy** (Leeds / Glasgow / London / All).

Two additional layers on top of the 9 tiles:

- **AI Insight Panel** — narrative summary + flagged risks + priorities, regenerated on filter change
- **Natural-language query box** — free-text Q&A over the dataset

Out of scope (explicit, not forgotten)

- Live integration with SuccessFactors / Kimble / Salesforce / Eploy — static data load only
- Real-time refresh
- Role-based / customisable views per user group — one view for all
- Audit trail, data governance controls, write-back
- Authentication / access control

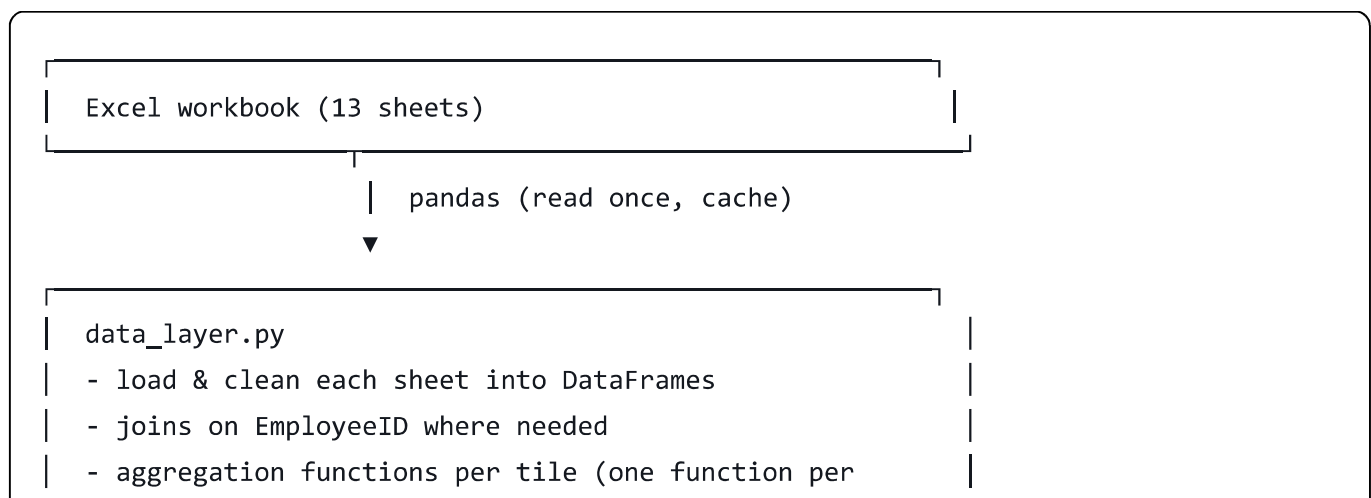
3. User stories

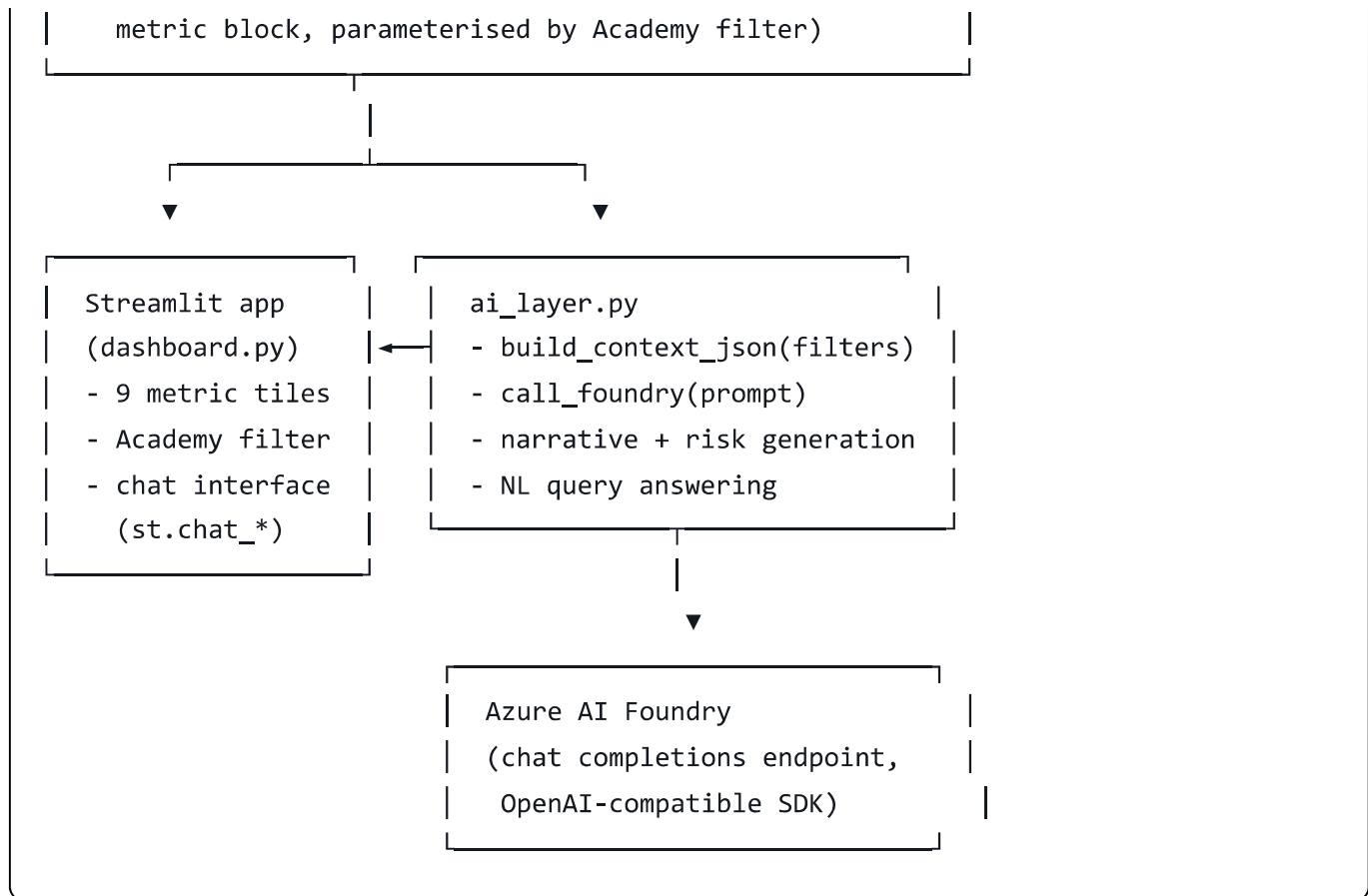
US1 — People team member: single view across all 8 lifecycle areas to manage workload and track impact. *Acceptance:* all 9 tiles render with numbers matching an independent pandas recompute.

US2 — Senior leader: automated, insight-led narrative instead of raw tables. *Acceptance:* AI panel returns a grounded summary + risks + priorities on every filter change; every claim traceable to a specific number in the data passed to the model.

US3 — Head of People: ask a direct question, get a direct answer. *Acceptance:* query box correctly answers at least 5 test questions spanning at least 4 different sheets.

4. Technical architecture





4.1 Dashboard platform: Streamlit

Recommended over alternatives for this build specifically because:

Option	Verdict for a 1-day build
Streamlit	Chosen. Pure Python, no separate frontend to build, hot-reloads on save, has native <code>st.chat_message</code> / <code>st.chat_input</code> components purpose-built for exactly the NL query box in US3, huge ecosystem of chart components (native + Plotly/Altair), fastest path from pandas DataFrame to rendered chart
Dash (Plotly)	More layout control and better suited to a polished production dashboard, but callback-based structure takes longer to wire up under time pressure, and chat UI has to be built by hand
Panel	Capable and Python-native, but smaller community/fewer ready-made examples to lean on mid-build
Gradio	Excellent for the chat piece specifically, weaker for multi-tile dashboard layouts — would need custom components for the metric grid

Streamlit wins on this brief because the two hardest requirements — "dashboard with multiple metric views" and "chat box" — are both first-class, low-code features of the same framework. Install: `pip install streamlit`.

4.2 AI model calls: Azure AI Foundry

Foundry exposes an **OpenAI-compatible endpoint**, so the standard `openai` Python SDK works directly — no proprietary SDK required for basic chat completions:

```
python

from openai import OpenAI
from azure.identity import DefaultAzureCredential, get_bearer_token_provider

token_provider = get_bearer_token_provider(
    DefaultAzureCredential(), "https://ai.azure.com/.default"
)

client = OpenAI(
    base_url="https://<YOUR-FOUNDRY-RESOURCE>.openai.azure.com/openai/v1/",
    api_key=token_provider,  # or a plain API key string if using key-based auth
)

def call_foundry(system_prompt: str, user_prompt: str) -> str:
    response = client.chat.completions.create(
        model="<your-deployment-name>",  # e.g. gpt-4o-mini, or your deployed model
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    return response.choices[0].message.content
```

Two auth options — pick whichever your Foundry resource is set up for:

- **Microsoft Entra ID** (`DefaultAzureCredential`, shown above) — recommended, no secrets to manage during the demo
- **API key** — simpler to wire up fast; pass the key string directly as `api_key=`

Deploy a small, fast model for this use case (e.g. a `gpt-4o-mini`-class deployment) — the workload is short-context summarisation and Q&A over a few KB of JSON, not heavy reasoning, so latency matters more than raw capability for a live demo.

5. Building the AI natural-language query chat box

This is the centrepiece demo feature. Two parts: the Streamlit chat UI, and the grounding strategy that stops it hallucinating.

5.1 Streamlit chat UI (mechanical part)

```
python
```

```

import streamlit as st

st.subheader("Ask People Pulse")

if "messages" not in st.session_state:
    st.session_state.messages = []

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

if question := st.chat_input("Ask a question about People data..."):
    st.session_state.messages.append({"role": "user", "content": question})
    with st.chat_message("user"):
        st.markdown(question)

    with st.chat_message("assistant"):
        answer = answer_query(question, current_filters=st.session_state.get("academy_
        st.markdown(answer)
    st.session_state.messages.append({"role": "assistant", "content": answer})

```

5.2 Grounding strategy — how `answer_query` avoids hallucination

The critical design decision: **don't let the model free-associate over the whole workbook.**
Two viable approaches, pick one for the day:

Approach A — Full-context stuffing (simplest, recommended for a 1-day build) Pre-aggregate all 13 sheets into a compact JSON summary (not raw rows — summary stats per sheet: counts, means, breakdowns by Academy/Department/Status) and pass the whole thing as context on every query. At this data volume (85 employees, low hundreds of rows per sheet) the aggregated JSON is small enough to fit comfortably in context every time — no retrieval infrastructure needed.

```

python

def answer_query(question: str, current_filters=None) -> str:
    context = build_context_json(current_filters)  # same aggregation used for the dashboard
    system_prompt = (
        "You are a People analytics assistant. Answer ONLY using the JSON data provided."
        "If the answer isn't in the data, say so explicitly. Cite the specific numbers."
        "Never invent figures not present in the JSON."
    )
    user_prompt = f"DATA:\n{context}\n\nQUESTION:\n{question}"
    return call_foundry(system_prompt, user_prompt)

```

Approach B — Function calling / tool use (more robust, more build time) Define the aggregation functions from `(data_layer.py)` as tools the model can call (e.g. `(get_bench_summary(academy))`, `(get_pay_review_status(academy))`), let Foundry's function-calling decide which to invoke based on the question, execute in Python, return the result to the model to phrase as an answer. More accurate on edge-case questions, but costs more build time to wire the tool schema — only attempt this if the dashboard + Approach A are done with time to spare.

Recommendation: build Approach A first. It satisfies US3's acceptance criteria at this data scale, and it reuses the exact same aggregation functions already built for the dashboard tiles — no duplicate logic.

5.3 Test questions to validate against before demo day

Pick at least one per sheet area so the 5-question acceptance bar in US3 is genuinely representative:

1. "Which academy has the highest ER case rate per head?" (*ER_Case_Log*)
2. "How many consultants are currently on the bench with a High risk flag?" (*Bench_Tracking*)
3. "What's the average time to hire from LinkedIn vs Employee Referral?" (*TA_Funnel*)
4. "How many FTC contracts are due to expire soon?" (*FTC_Tracking*)
5. "What's the pay review approval rate in Leeds?" (*Pay_Reviews*)

Run these manually against the built context JSON before the live demo — if any answer is wrong, it's very likely a data-layer aggregation bug, not a prompting bug, so check `(data_layer.py)` output first.

6. Build steps

Step 1 — Data layer (Python / pandas)

1. Load all 13 sheets from the workbook
2. Write one aggregation function per dashboard tile (9 functions), each accepting an `(academy)` filter parameter and returning a small dict/JSON-serialisable summary
3. Write `(build_context_json(filters))` that calls all 9 functions and merges the output — this becomes both the dashboard's data source and the chat box's grounding context
4. Unit-check each function's output against a manual `(.groupby())` in a notebook before moving on

Step 2 — Dashboard shell (Streamlit)

5. `(st.set_page_config)` + page title + Academy filter (`(st.selectbox)`)
6. Lay out 9 tiles in a grid (`(st.columns)`) — each tile calls its data-layer function and renders a metric/chart (`(st.metric)`, `(st.bar_chart)`, or Plotly if richer charts are needed)

7. Verify every tile updates correctly when the Academy filter changes

Step 3 — Azure Foundry connection

8. Confirm Foundry resource, deployment name, and auth method (Entra ID vs API key) with whoever owns the Azure subscription — do this early, first thing in the morning, since auth setup is the most likely blocker
9. Write `call_foundry()` and smoke-test with a trivial prompt before building anything on top of it

Step 4 — AI Insight Panel

10. Write the system prompt for the narrative (tone: exec-facing, concise, grounded — reuse the guardrail language from §5.2)
11. Wire it to regenerate on Academy filter change
12. Sanity-check output across all 4 filter states (All, Leeds, Glasgow, London) for grounding accuracy

Step 5 — NL query chat box

13. Build the `st.chat_input` / `st.chat_message` UI (§5.1)
14. Wire `answer_query()` using Approach A (§5.2)
15. Run the 5 test questions (§5.3), fix any data-layer issues found

Step 6 — Polish & demo prep

16. Visual pass: consistent chart colours/labels, tidy tile titles
17. Prepare the 2-minute demo script (below)
18. Have a fallback screenshot/recording in case of live Foundry latency or connectivity issues on the day

7. Day plan (indicative timings)

Time	Block
09:00–09:30	Confirm Foundry access/auth; lock deployment name
09:30–11:30	Data layer: load, clean, 9 aggregation functions
11:30–13:00	Dashboard shell: filter + 9 tiles rendering correctly
13:00–13:30	Lunch
13:30–15:00	AI Insight Panel: prompt, wiring, grounding checks
15:00–16:30	NL query chat box: UI + Approach A + test questions

Time	Block
16:30-17:00	Polish, demo script, fallback recording

8. Success measures (demo-day)

- All 9 tiles render correctly for every Academy filter state, verified against manual recompute
- AI narrative changes meaningfully with filter, never repeats verbatim across states
- All 5 test questions answered correctly live
- Every number shown or spoken can be traced back to source data on request

9. Demo script (2 minutes)

1. Open on "All" — walk the 9-tile lifecycle view (15s: *"this used to mean five spreadsheets and a afternoon of VLOOKUPS"*)
2. Filter to one academy — AI panel regenerates narrative live, call out one specific flagged risk
3. Ask a live, unscripted question in the chat box, get a correct grounded answer
4. Close: *"this proves the concept — multi-system integration and governance are the next phase, not the hard part."*